```
!pip install pandas
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.0.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2026.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0
```

```
import pandas as pd
```

```
df = pd.read_csv("/content/movies.csv")
```

Q1.How do we load the dataset and see the first 5 rows?

```
df.head()
```

	id	name	year	rating	certificate	duration	genre	votes	gross_income	directors_id	d
0	tt0068646	The Godfather	1972	9.2	R	175 min	Crime, Drama	1798749	134,966,411	nm0000338	
1	tt5113044	Minions: The Rise of Gru	2022	7.3	PG	87 min	Animation, Adventure, Comedy	1,220	134,966,411	nm0049633,nm1556070,nm3646390	I Ab
2	tt7657566	Death on the Nile	2022	6.3	PG-13	127 min	Crime, Drama, Mystery	121,063	134,966,411	nm0000110	K
3	tt8115900	The Bad Guys	2022	6.9	PG	100 min	Animation, Adventure, Comedy	23,090	134,966,411	nm2010048	
4	tt10809742	Collision	2022	3.8	TV-MA	99 min	Crime, Drama, Thriller	656	134,966,411	nm1926494	

Next steps: Generate code with df | New interactive sheet

```
df.tail()    #To get the last 5 rows
```

	id	name	year	rating	certificate	duration	genre	votes	gross_income	director
19803	tt0459423	Kahaani Gudiya Ki...: True Story of a Woman	2008	6.2	Not Rated	112 min	Biography, Drama	41	44,435	nm190
19804	tt0119621	Mark Twain's America in	1998	6.5	G	52 min	Documentary, Biography	48	2,281,741	nm052
19805	tt0168950	Let It Come Down: The Life of Paul Bowles	1998	6.9	Not Rated	75 min	Documentary, Biography	125	9,188	nm004
19806	tt0267563	Hybrid	2000	7.4	Not Rated	92 min	Documentary, Biography	146	159,403	/name/nm0566144/,/name/nm108
19807	tt0116481	Hands on a Hardbody: The Documentary	1997	7.6	PG	98 min	Documentary, Comedy, Drama	2167	492,876	nm008

```
df.sample(5)   #To get sample data
```

	id	name	year	rating	certificate	duration	genre	votes	gross_income	directors_id	directors_name	
14949	tt1610525	Chuck	2016	6.5	R	98 min	Biography, Drama, Sport	6347	320,725	nm0265852	Philippe Falardeau	nm
12640	tt0160644	Passion of Mind	2000	5.5	PG-13	105 min	Drama, Mystery, Romance	3427	769,009	nm0075626	Alain Berliner	nm
14902	tt0118570	Air Bud	1997	5.2	PG	98 min	Comedy, Drama, Family	17877	24,629,916	nm0001747	Charles Martin Smith	nm
16250	tt0053271	The Shaggy Dog	1959	6.4	Approved	104 min	Comedy, Family, Fantasy	5083	100,935	nm0059106	Charles Barton	nm
19184	tt0326976	Intoxicating	2003	4.5	Not Rated	109 min	Drama, Thriller	390	192,000	nm0202999	Mark David	nm

Q2.What are the datatypes of each columns?

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19808 entries, 0 to 19807
Data columns (total 14 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   id              19808 non-null  object
 1   name            19808 non-null  object
 2   year            19808 non-null  int64
 3   rating          19808 non-null  float64
 4   certificate     19808 non-null  object
```

```
 5   duration        19808 non-null  object
 6   genre           19808 non-null  object
 7   votes           19808 non-null  object
 8   gross_income    19808 non-null  object
 9   directors_id    19808 non-null  object
 10  directors_name  19808 non-null  object
 11  stars_id        19808 non-null  object
 12  stars_name      19808 non-null  object
 13  description     19808 non-null  object
dtypes: float64(1), int64(1), object(12)
memory usage: 2.1+ MB
```

```
df.describe()  #Gives statistical value
```

	year	rating
count	19808.000000	19808.000000
mean	1995.546597	6.041973
std	22.801725	1.206831
min	1894.000000	1.100000
25%	1983.000000	5.300000
50%	2003.000000	6.200000
75%	2014.000000	6.900000
max	2022.000000	9.700000

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
df.isna().sum()
```

	0
id	0
name	0
year	0
rating	0
certificate	0
duration	0
genre	0
votes	0
gross_income	0
directors_id	0
directors_name	0
stars_id	0
stars_name	0
description	0

dtype: int64

Q3.How do we clean the 'duration' column ? (e.g., '175min' - 175)

```
df["duration"] = df["duration"].str.replace("min","")
```

```
df["duration"] = df["duration"].str.replace(",","")
```

```
df["duration"] = df["duration"].astype(int)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19808 entries, 0 to 19807
Data columns (total 14 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   id              19808 non-null  object
 1   name            19808 non-null  object
 2   year            19808 non-null  int64
 3   rating          19808 non-null  float64
 4   certificate     19808 non-null  object
 5   duration        19808 non-null  int64
 6   genre           19808 non-null  object
 7   votes           19808 non-null  object
 8   gross_income    19808 non-null  object
 9   directors_id    19808 non-null  object
 10  directors_name  19808 non-null  object
 11  stars_id        19808 non-null  object
 12  stars_name      19808 non-null  object
 13  description     19808 non-null  object
dtypes: float64(1), int64(2), object(11)
memory usage: 2.1+ MB
```

```
df["duration"].mean()
```

```
np.float64(101.77796849757674)
```

Q4.How do we clean the 'votes' and 'gross_income' column?

```
df["votes"] =  df["votes"].str.replace(",","")
```

```
df["votes"] = df["votes"].astype(int)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19808 entries, 0 to 19807
Data columns (total 14 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   id              19808 non-null  object
 1   name            19808 non-null  object
 2   year            19808 non-null  int64
 3   rating          19808 non-null  float64
 4   certificate     19808 non-null  object
 5   duration        19808 non-null  int64
 6   genre           19808 non-null  object
 7   votes           19808 non-null  int64
 8   gross_income    19808 non-null  object
 9   directors_id    19808 non-null  object
 10  directors_name  19808 non-null  object
 11  stars_id        19808 non-null  object
 12  stars_name      19808 non-null  object
 13  description     19808 non-null  object
dtypes: float64(1), int64(3), object(10)
memory usage: 2.1+ MB
```

```
df["gross_income"] = df["gross_income"].str.replace(",","")
```

```
df["gross_income"] = df["gross_income"].str.replace("$","")
```

```
df["gross_income"] = df["gross_income"].str.replace("M","")
```

```
df["gross_income"] = df["gross_income"].astype(float)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19808 entries, 0 to 19807
Data columns (total 14 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   id              19808 non-null  object
 1   name            19808 non-null  object
 2   year            19808 non-null  int64
 3   rating          19808 non-null  float64
```

```
 4   certificate     19808 non-null  object
 5   duration        19808 non-null  int64
 6   genre           19808 non-null  object
 7   votes           19808 non-null  int64
 8   gross_income    19808 non-null  float64
 9   directors_id    19808 non-null  object
 10  directors_name  19808 non-null  object
 11  stars_id        19808 non-null  object
 12  stars_name      19808 non-null  object
 13  description     19808 non-null  object
dtypes: float64(2), int64(3), object(9)
memory usage: 2.1+ MB
```

Q5.How many missing values are in each columns?

```
df.isnull().sum()
```

	0
id	0
name	0
year	0
rating	0
certificate	0
duration	0
genre	0
votes	0
gross_income	0
directors_id	0
directors_name	0
stars_id	0
stars_name	0
description	0

dtype: int64

Q6.Find the "Hidden Gems"(High rated , Low votes)

```
df.describe()
```

	year	rating	duration	votes	gross_income
count	19808.000000	19808.000000	19808.000000	1.980800e+04	1.980800e+04
mean	1995.546597	6.041973	101.777968	3.694303e+04	1.682198e+07
std	22.801725	1.206831	43.178721	1.129755e+05	4.642034e+07
min	1894.000000	1.100000	1.000000	5.000000e+00	0.000000e+00
25%	1983.000000	5.300000	90.000000	1.058000e+03	1.196010e+05
50%	2003.000000	6.200000	98.000000	4.233000e+03	1.158192e+06
75%	2014.000000	6.900000	111.000000	2.045350e+04	1.243278e+07
max	2022.000000	9.700000	5220.000000	2.574832e+06	9.366622e+08

```
df[(df["rating"] > 6.9) & (df["votes"] < 1058)]
```

	id	name	year	rating	certificate	duration	genre	votes	gross_income	directo
3423	tt4106286	Tiny Detectives	2014	7.2	TV-14	2	Short, Comedy, Crime	536	1501277.0	nm01
3502	tt0042953	Shakedown	1950	7.0	Approved	80	Crime, Drama, Film-Noir	346	6310.0	nm06
3522	tt0070552	High Crime	1973	7.0	R	100	Crime, Drama, Thriller	885	1404.0	nm01
3594	tt0246879	Qurbani	1980	7.0	Not Rated	157	Action, Crime, Drama	886	398420.0	nm04
3980	tt13634096	The Bacon Hair	2020	8.5	TV-14	76	Animation, Action, Adventure	90	852206.0	nm97
...	...	...	...	...	...	...	...	...	...	
19799	tt1486548	Ahead of Time: The Extraordinary Journey of Ru...	2009	7.9	Not Rated	73	Documentary, Biography	77	33024.0	nm07
19800	tt0044917	Amazing Monsieur Fabre	1951	7.0	Approved	90	Biography, Comedy, Drama	64	33024.0	nm02
19801	tt1334479	Yoo-Hoo, Mrs. Goldberg	2009	7.4	Unrated	92	Documentary, Biography	236	1133662.0	nm04
19802	tt1518255	Raw Faith	2010	8.3	Not Rated	94	Documentary, Biography, Drama	71	1486.0	/name/nm0927286/,/name/nm36
19806	tt0267563	Hybrid	2000	7.4	Not Rated	92	Documentary, Biography	146	159403.0	/name/nm0566144/,/name/nm10

525 rows × 14 columns

Q7.Find the "Overhyped" movies(Low Rating, High Gross)

```
df.describe()
```

	year	rating	duration	votes	gross_income
count	19808.000000	19808.000000	19808.000000	1.980800e+04	1.980800e+04
mean	1995.546597	6.041973	101.777968	3.694303e+04	1.682198e+07
std	22.801725	1.206831	43.178721	1.129755e+05	4.642034e+07
min	1894.000000	1.100000	1.000000	5.000000e+00	0.000000e+00
25%	1983.000000	5.300000	90.000000	1.058000e+03	1.196010e+05
50%	2003.000000	6.200000	98.000000	4.233000e+03	1.158192e+06
75%	2014.000000	6.900000	111.000000	2.045350e+04	1.243278e+07
max	2022.000000	9.700000	5220.000000	2.574832e+06	9.366622e+08

```
df[(df["rating"] < 5.3) & (df["gross_income"] > 12432780)]
```

	id	name	year	rating	certificate	duration	genre	votes	gross_income	directors_id	directors_name
4	tt10809742	Collision	2022	3.8	TV-MA	99	Crime, Drama, Thriller	656	134966411.0	nm1926494	Fabien Martorell
18	tt11656220	Midnight in the Switchgrass	2021	4.4	R	99	Action, Crime, Thriller	6827	107928762.0	nm0256542	Randall Emmett
37	tt6749318	Speed Kills	2018	4.2	R	102	Action, Crime, Drama	3381	130742922.0	nm9928566	Jodi Scurfield
40	tt5433138	F9: The Fast Saga	2021	5.2	PG-13	143	Action, Crime, Thriller	125648	173202780.0	nm0510912	Justin Lin
47	tt1551614	Blowback	2022	4.4	R	93	Action, Crime, Thriller	199	74283625.0	nm0847749	Tibor Takács
...	...	...	...	...	...	...	...	...	...	...	...
18575	tt1389139	When the Bough Breaks	2016	5.1	PG-13	107	Drama, Thriller	6974	29703843.0	nm0143984	Jon Cassar
18615	tt8170298	John Henry	2020	3.6	R	91	Drama, Thriller	2368	21082165.0	nm3112158	Will Forbes
18616	tt6469960	An Imperfect Murder	2017	3.2	R	71	Thriller	1666	15156200.0	nm0864812	James Toback
18717	tt1859573	Pretty Obsession	2012	4.2	Not Rated	89	Thriller	193	12974636.0	nm0062341	Michael Baumgarten
19592	tt0074562	Gable and Lombard	1976	5.1	R	131	Biography, Drama, Romance	601	19161363.0	nm0002089	Sidney J. Furie

733 rows × 14 columns

Q8.What are the unique certificates in the dataset?

```
df["certificate"].unique()
```

```
array(['R', 'PG', 'PG-13', 'TV-MA', 'Not Rated', 'Approved', 'TV-14',
       'NC-17', 'Unrated', 'G', 'Passed', 'GP', 'X', 'M/PG', 'M', 'TV-PG',
       'TV-G', '12', 'TV-13', 'TV-Y7-FV', 'MA-17', 'AO', '18', 'R-15',
       'R-12', 'R-18', 'PG-12', '6+', 'TV-Y7', 'TV-Y', 'MA-13'],
      dtype=object)
```

Q9.How many unique certificates are there?

```
df["certificate"].nunique()
```

```
31
```

Q10.How many movies were released in each year?

```
df.groupby("year")["id"].count().sort_values(ascending = False)
```

year	id
2018	736
2017	714
2016	665
2019	657
2014	640
...	...
1898	1
1919	1
1903	1
1917	1
1894	1

114 rows × 1 columns

dtype: int64

```
df["year"].value_counts().sort_index(ascending = False)
```

year	count
2022	174
2021	470
2020	467
2019	657
2018	736
...	...
1913	3
1903	1
1902	2
1898	1
1894	1

114 rows × 1 columns

dtype: int64

Q11. Which are the top 5 most common genres?

```
df.head()
```

	id	name	year	rating	certificate	duration	genre	votes	gross_income	directors_id	d
0	tt0068646	The Godfather	1972	9.2	R	175	Crime, Drama	1798749	134966411.0	nm0000338	
1	tt5113044	Minions: The Rise of Gru	2022	7.3	PG	87	Animation, Adventure, Comedy	1220	134966411.0	nm0049633,nm1556070,nm3646390	I Ab
2	tt7657566	Death on the Nile	2022	6.3	PG-13	127	Crime, Drama, Mystery	121063	134966411.0	nm0000110	K
3	tt8115900	The Bad Guys	2022	6.9	PG	100	Animation, Adventure, Comedy	23090	134966411.0	nm2010048	
4	tt10809742	Collision	2022	3.8	TV-MA	99	Crime, Drama, Thriller	656	134966411.0	nm1926494	

Next steps: Generate code with df New interactive sheet

```
x = df["genre"].str.split(",").explode()

x = x.str.strip()
```

```
x.value_counts().sort_values(ascending = False).head()
```

genre	count
Drama	9655
Action	5290
Crime	5244
Comedy	4641
Thriller	4231

dtype: int64

Q12. Which directors have directed the most movies?

```
df.head()
```

	id	name	year	rating	certificate	duration	genre	votes	gross_income	directors_id	d
0	tt0068646	The Godfather	1972	9.2	R	175	Crime, Drama	1798749	134966411.0	nm0000338	
1	tt5113044	Minions: The Rise of Gru	2022	7.3	PG	87	Animation, Adventure, Comedy	1220	134966411.0	nm0049633,nm1556070,nm3646390	I Ab
2	tt7657566	Death on the Nile	2022	6.3	PG-13	127	Crime, Drama, Mystery	121063	134966411.0	nm0000110	K
3	tt8115900	The Bad Guys	2022	6.9	PG	100	Animation, Adventure, Comedy	23090	134966411.0	nm2010048	
4	tt10809742	Collision	2022	3.8	TV-MA	99	Crime, Drama, Thriller	656	134966411.0	nm1926494	

Next steps: Generate code with df | New interactive sheet

```
y = df["directors_name"].str.split(",").explode()
```

```
y.value_counts().sort_values(ascending = False).head()
```

directors_name	count
Clint Eastwood	38
Alfred Hitchcock	38
Henry Hathaway	31
John Ford	31
Raoul Walsh	31

dtype: int64

Q13. Do longer movies get better ratings?

```
df[["duration","rating"]].corr()
```

	duration	rating
duration	1.000000	0.130112
rating	0.130112	1.000000

```
# vote VS gross_income

df[["votes","gross_income"]].corr()
```

	votes	gross_income
votes	1.000000	0.632683
gross_income	0.632683	1.000000

Q14. Which genre has the highest average rating?

```
df[["genre","rating"]]
```

	genre	rating
0	Crime, Drama	9.2
1	Animation, Adventure, Comedy	7.3
2	Crime, Drama, Mystery	6.3
3	Animation, Adventure, Comedy	6.9
4	Crime, Drama, Thriller	3.8
...	...	...
19803	Biography, Drama	6.2
19804	Documentary, Biography	6.5
19805	Documentary, Biography	6.9
19806	Documentary, Biography	7.4
19807	Documentary, Comedy, Drama	7.6

19808 rows × 2 columns

```
df2 = df.assign(genre = df["genre"].str.split(", ")).explode("genre")
```

```
df.head(2)
```

	id	name	year	rating	certificate	duration	genre	votes	gross_income	directors_id	di
0	tt0068646	The Godfather	1972	9.2	R	175	Crime, Drama	1798749	134966411.0	nm0000338	
1	tt5113044	Minions: The Rise of Gru	2022	7.3	PG	87	Animation, Adventure, Comedy	1220	134966411.0	nm0049633,nm1556070,nm3646390	K Abl

Next steps: Generate code with df | New interactive sheet

```
df2.head()
```

	id	name	year	rating	certificate	duration	genre	votes	gross_income	directors_id	dir
0	tt0068646	The Godfather	1972	9.2	R	175	Crime	1798749	134966411.0	nm0000338	
0	tt0068646	The Godfather	1972	9.2	R	175	Drama	1798749	134966411.0	nm0000338	
1	tt5113044	Minions: The Rise of Gru	2022	7.3	PG	87	Animation	1220	134966411.0	nm0049633,nm1556070,nm3646390	Ky Able
1	tt5113044	Minions: The Rise of Gru	2022	7.3	PG	87	Adventure	1220	134966411.0	nm0049633,nm1556070,nm3646390	Ky Able
1	tt5113044	Minions: The Rise of Gru	2022	7.3	PG	87	Comedy	1220	134966411.0	nm0049633,nm1556070,nm3646390	Ky Able

Next steps: Generate code with df2 New interactive sheet

```
df2.groupby("genre")["rating"].mean().sort_values(ascending = False).head()
```

genre	rating
Documentary	7.185514
Reality-TV	7.100000
Short	7.063351
Biography	6.860367
Film-Noir	6.821493

dtype: float64

Q15. Which genre makes the most money (Avg Gross)?

```
df2.groupby("genre")["gross_income"].mean().sort_values(ascending = False).head()
```

genre	gross_income
Animation	4.496143e+07
Adventure	4.201662e+07
Sci-Fi	3.358844e+07
Fantasy	2.804439e+07
Family	2.800047e+07

dtype: float64

Q16. Who is the highest-grossing director of all time (in this dataset)?

```
df.groupby("directors_name")["gross_income"].sum().sort_values(ascending = False).head()
```

directors_name	gross_income
Steven Spielberg	4.247700e+09
Michael Bay	2.703051e+09
Anthony Russo,Joe Russo	2.262877e+09
J.J. Abrams	2.199407e+09
Peter Jackson	2.156934e+09

dtype: float64

Q17. How does Certificate affect Gross Income?

```
df.groupby("certificate")["gross_income"].mean().sort_values(ascending = False).head()
```

certificate	gross_income
GP	6.221153e+07
PG-13	4.571531e+07
MA-17	3.806747e+07
R-18	3.474929e+07
PG	2.997915e+07

dtype: float64

Q18. Are ratings improving over the years?

```
df[["year","rating"]].corr()
```

	year	rating
year	1.000000	-0.146652
rating	-0.146652	1.000000

```
df.groupby("year")["rating"].mean().sort_index()
```

year	rating
1894	5.300000
1898	7.500000
1902	7.400000
1903	6.400000
1913	6.400000
...	...
2018	5.935598
2019	5.914460
2020	5.577516
2021	5.741064
2022	5.481609

114 rows × 1 columns

dtype: float64

VISUALIZATION

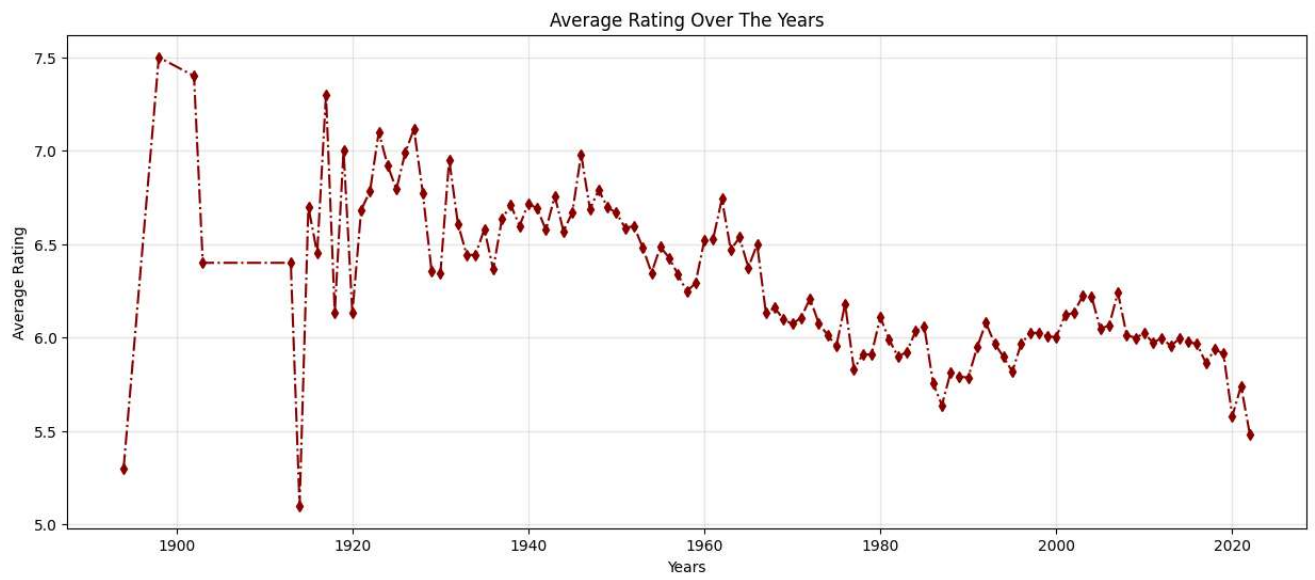
```
import matplotlib.pyplot as plt
```

Q1. Plot the average movie rating over the years (Line Chart)

```
roy = df.groupby("year")["rating"].mean().sort_index()
```

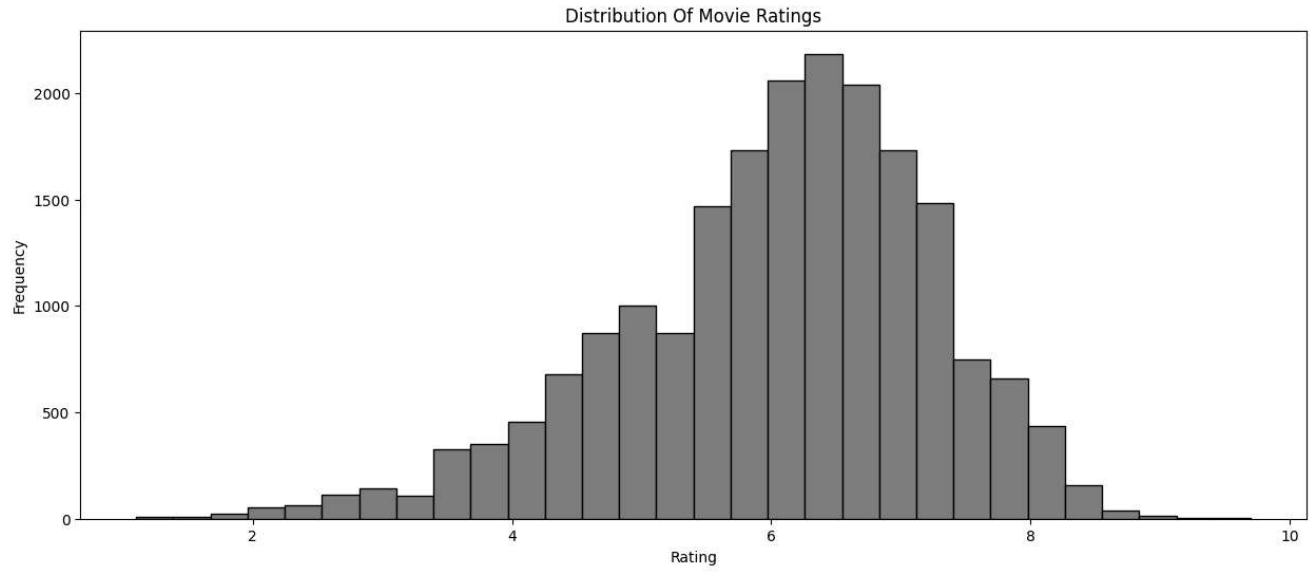
```
plt.figure(figsize = (15,6))

plt.plot(roy.index, roy.values, color = "darkred", marker = "d", markersize = 5, linestyle = "-.")

plt.title("Average Rating Over The Years")
plt.xlabel("Years")
plt.ylabel("Average Rating")

plt.grid(alpha = 0.3)
```

```
plt.show()
```



Q3. Plot the distribution of movie ratings (Histogram using Matplotlib)

```
plt.figure(figsize = (15,6))

plt.hist(df["rating"], color = "gray", edgecolor = "black", bins = 30)

plt.title("Distribution Of Movie Ratings")
plt.xlabel("Rating")
plt.ylabel("Frequency")

plt.show()
```



Q5. Which are the top 10 most frequent actors? (Horizontal Bar Chart)

```
x = df["stars_name"].str.split(",").explode()
```

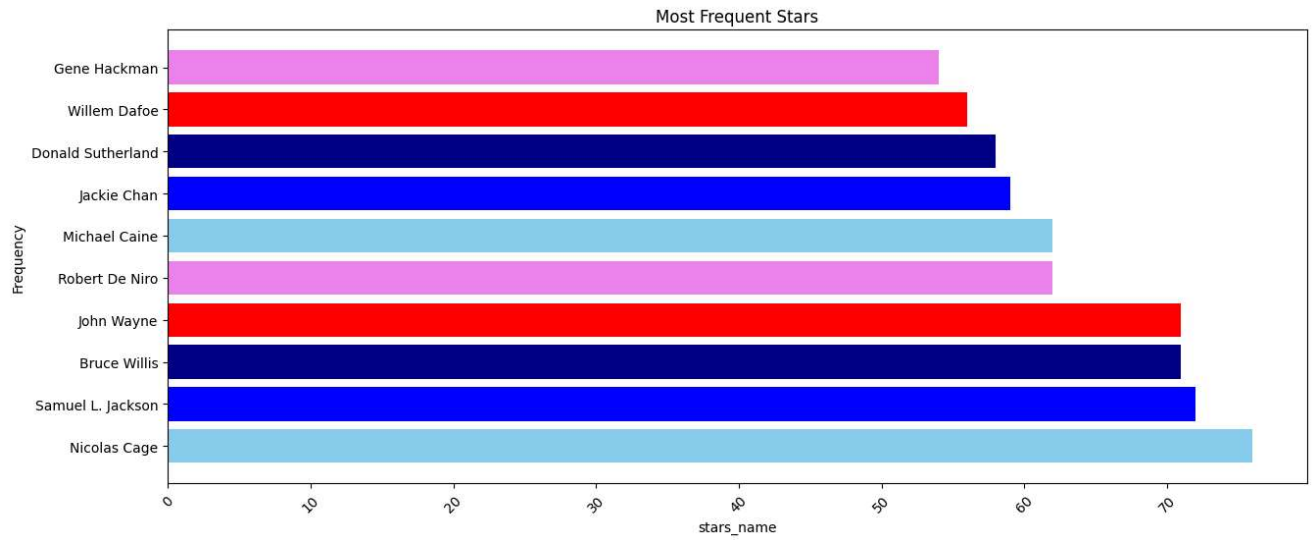
```
top_10 = x.value_counts().sort_values(ascending = False).head(10)
```

```
top_10
```

stars_name	count
Nicolas Cage	76
Samuel L. Jackson	72
Bruce Willis	71
John Wayne	71
Robert De Niro	62
Michael Caine	62
Jackie Chan	59
Donald Sutherland	58
Willem Dafoe	56
Gene Hackman	54

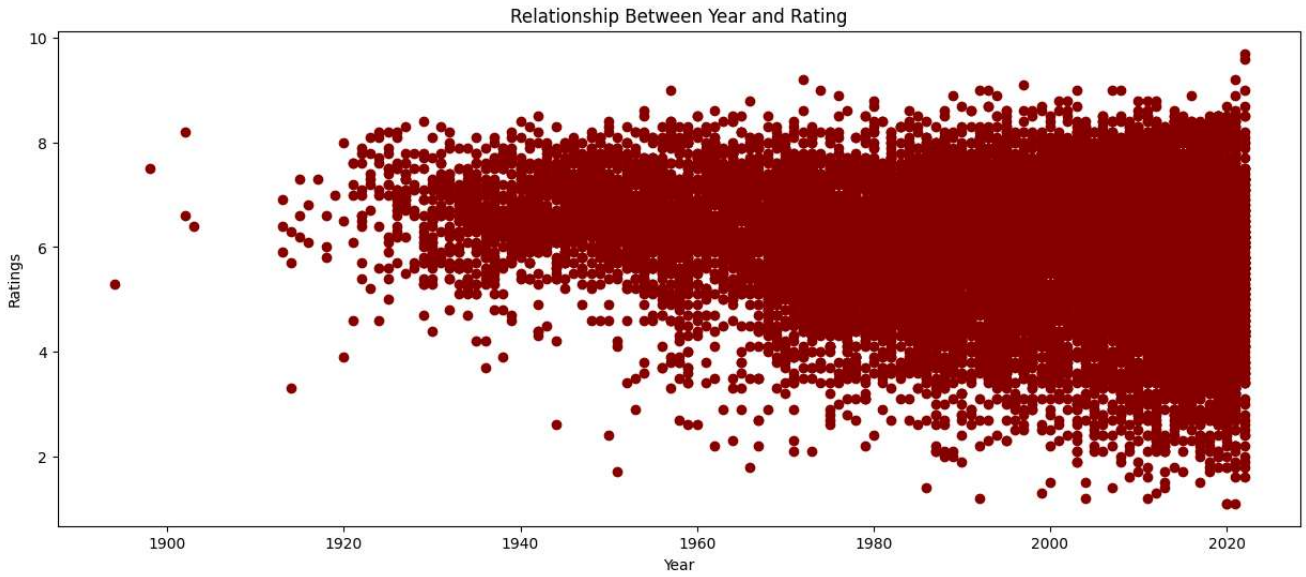
dtype: int64

```
plt.figure(figsize = (15,6))

plt.barh(top_10.index, top_10.values, color = ["skyblue", "blue", "darkblue","red", "violet"])

plt.title("Most Frequent Stars")
plt.xlabel("stars_name")
plt.ylabel("Frequency")

plt.xticks(rotation = 45)

plt.show()
```



Q6. Plot the relationship between year and rating (Scatter Plot)

```
plt.figure(figsize = (15,6))

plt.scatter(df["year"], df["rating"], color = "darkred")

plt.title("Relationship Between Year and Rating")
plt.xlabel("Year")
plt.ylabel("Ratings")

plt.show()
```



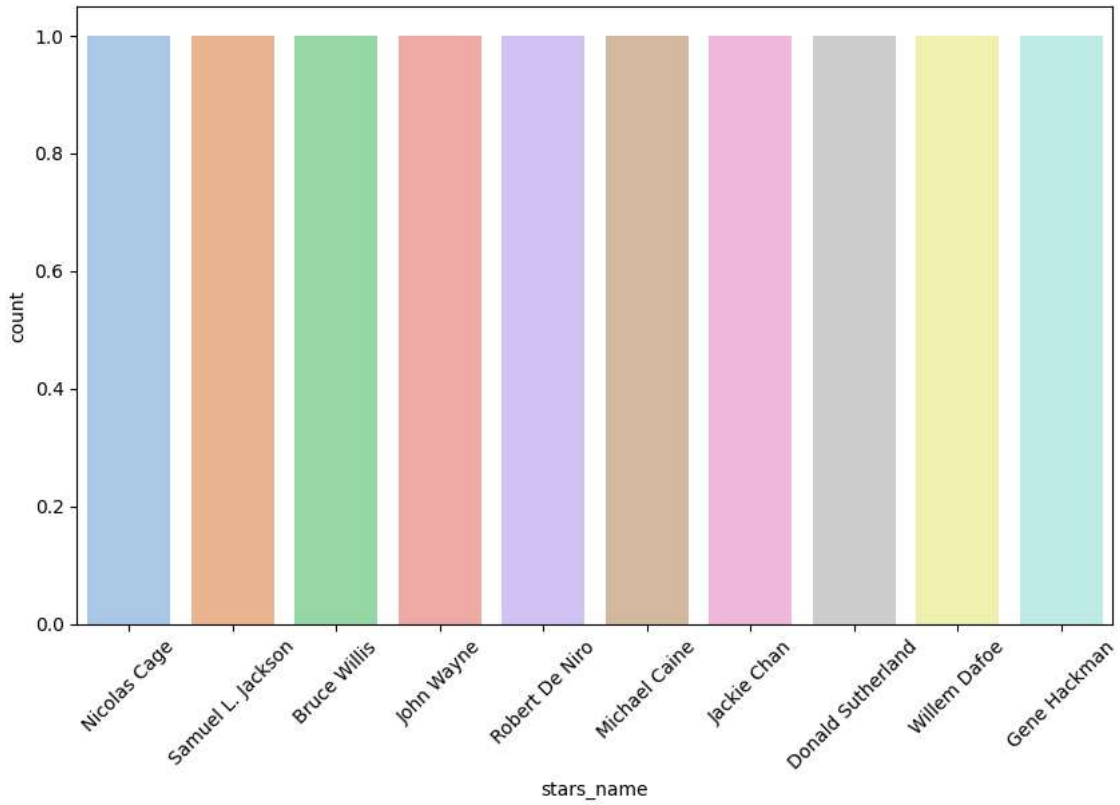

```
import seaborn as sns
```

Q7. Plot the count of movies for the top actors (Count Plot using Seaborn)

```
top_10 = df["stars_name"].str.split(",").explode()

top_10 = top_10.value_counts().sort_values(ascending = False).head(10)
```

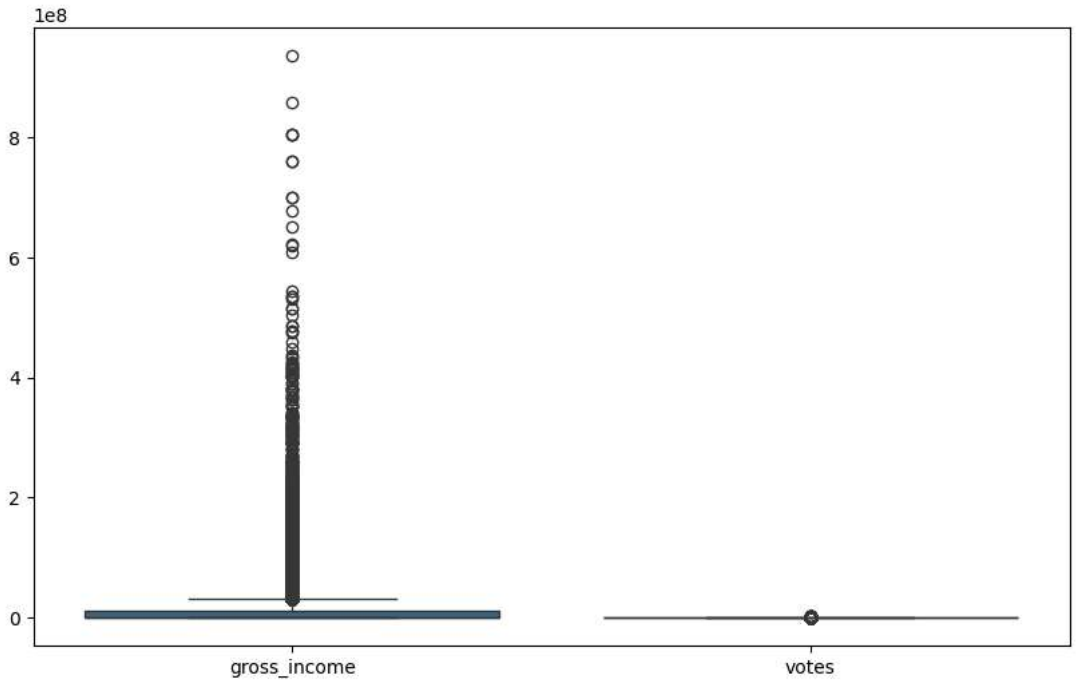
```
plt.figure(figsize = (10,6))

sns.countplot(top_10, palette = "pastel")
plt.xticks(rotation = 45)
plt.show()
```

```
/tmp/ipykernel_18428/7136572.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set

  sns.countplot(top_10, palette = "pastel")
```

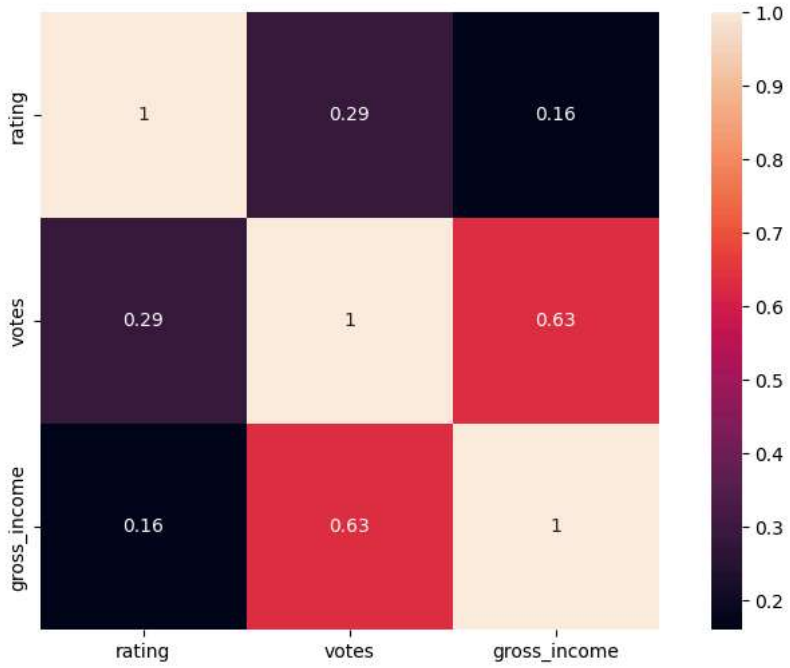


Q8. Plot the distribution of Gross Income and Votes (Box Plot)

```
plt.figure(figsize = (10,6))

sns.boxplot(df[["gross_income","votes"]], palette = "viridis")
plt.show()
```



```
plt.figure(figsize = (10,6))

sns.heatmap(df[["rating", "votes", "gross_income"]].corr(), annot = True, square = True)
plt.show()
```



Q10. Plot the correlation of all numeric columns (Full Heatmap)

```
plt.figure(figsize = (10,6))

sns.heatmap(df.corr(numeric_only = True), annot = True, square = True, cmap = "Blues")
plt.show()
```